Probing-Secure EqSat

Shreyash Panigrahy
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore 632014, Tamil Nadu, India
shreyash.panigrahy2024@vitstudent.ac.in

Sourajyoti Basak
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore 632014, Tamil Nadu, India
sourajyoti.basak2024@vitstudent.ac.in

J. Nandhakishor
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore 632014, Tamil Nadu, India
jalagam.nandha2024@vitstudent.ac.in

Krishnaraj N
Dept. of Database Systems, School of CSE (SCOPE)
Vellore Institute of Technology
Vellore 632014, Tamil Nadu, India
krishnaraj.n@vit.ac.in

***Abstract*—A chip that holds secret keys can give them away through its power use. Masking, the standard defence, splits each secret into random shares so that no single wire reveals anything, but masked circuits are large, and handing them to a compiler for optimisation is tempting. We ask whether equality saturation (EqSat), which collects every equivalent version of a program and then picks the cheapest, can do this safely. Left alone, it cannot. Starting from the textbook 8-gate masked AND gadget of Ishai, Sahai and Wagner (ISW), a cost-only optimiser picks a 7-gate version that exposes a secret on one wire; with two shares this happens for this gadget every time, because each gate-saving rewrite we enumerated puts a secret back together. Checking security when the final circuit is chosen repairs the problem for the price of one gate. The tricks that masking designers actually rely on, such as refreshes and gadget swaps, cannot be stated as equalities in a standard wire-level e-graph. We therefore build a second e-graph over secret values, and we show that choosing one circuit per class can be unsafe there. Two safe ways of choosing follow: checking every candidate, and a method based on probe-isolating non-interference (PINI) that certifies a 32-bit row of the Keccak χ map, with 2^{128} assignments of shares and random bits, in 0.05 s. We package the work as seqc, a small compiler from Boolean programs to masked Verilog, and every security verdict about our own circuits was confirmed by an independent verifier, with zero disagreements (more than 400,000 wire checks, most of them from enumerating every extraction of one small example). On twelve small programs, seqc needs no more AND gadgets than AGEMA and COMPRESS on ten, and when its logic is given to COMPRESS the glitch-robust design gets smaller on four programs and larger on three. The scope is first order, two shares and glitch-free output.**

***Index Terms*—masking, side-channel security, probing security, equality saturation, e-graphs, compiler optimisation, glitches, PINI**

I. Introduction

Breaking the mathematics of a cipher is not the only way to recover its key. A chip's power draw and electromagnetic field depend on the values moving through its wires, so an attacker with suitable equipment can, in effect, read a single wire. We call this a *probe*.

The standard defence is *masking*. A secret is never stored directly: it is split into random *shares*, and only all of them together reveal it, so any single wire looks like coin flips whatever the secret is. The protection is costly. Every AND gate must be replaced by a *gadget*, which adds gates and fresh random bits.

Handing masked circuits to an optimiser is the obvious next step, and it runs into a problem. An optimiser promises only to preserve a program's *meaning*, which is not enough for masked hardware: two circuits can compute exactly the same output and still differ in security, because security depends on the wires in between.

We study one widely used technique, equality saturation [1], [2]. It stores every equivalent version of a program in a data structure called an *e-graph* and then extracts the cheapest one. The question is a narrow one:

> **Research question.** Can an equality-saturation optimiser make masked circuits cheaper without making them leak?

We expected the answer to be "not by default" and split the question into four parts. Table I gives short answers.

The security ideas come from hardware, but the questions are compiler questions. Which rewrites are safe? How should an optimiser choose between equal versions? What should the cost model count, and what must the intermediate representation remember? To study them in practice we also built a

TABLE I
Our Four Questions and Short Answers

RQ1	*Does a cost-only optimiser keep the circuit secure?* **No.** It swaps the secure 8-gate gadget for a 7-gate one that leaks (Sec. III).
RQ2	*Can we fix it by checking security when the circuit is chosen?* **Yes**, for one extra gate. On a larger example we found no cheaper secure circuit (Sec. IV).
RQ3	*Can a standard e-graph express the tricks designers use?* **No.** We build a second e-graph that can, with a new way of choosing (Secs. V–VI).
RQ4	*Does this work with timing glitches and bigger circuits?* **Partly.** Registers are kept under glitches on gadgets and chains, and a 32-bit Keccak χ row is certified in the glitch-free model (Sec. VII-B).

complete compiler, from a small source language down to Verilog (Sec. IX).

A. What is new here

Existing tools fall into two groups. *Security tools*, such as masking compilers and verifiers, mostly keep the circuit's structure fixed: they choose gadgets, place registers or check a finished design, and do not rewrite the logic (a recent evolutionary search over gadget netlists is the exception, see Sec. XII). *Optimisers*, such as synthesis tools and e-graph engines, do rewrite the logic but know nothing about security. Our work combines the optimiser's freedom to rewrite with the security tools' checks. Our contributions are:

1) **We show the optimiser breaks masking, and explain why.** For the two-share ISW gadget, the cheaper circuit leaks *because* every gate-saving rewrite we enumerated recombines a secret.
2) **A simple fix.** Keep every rewrite rule, but refuse leaking wires when choosing the final circuit. This costs one gate.
3) **A limit of standard e-graphs.** The designers' main trick, re-sharing, is not the kind of equality an e-graph is allowed to use.
4) **A second e-graph** that works with secret values instead of wires, and a proof that the usual way of choosing from it is unsafe.
5) **Two safe ways to choose**: one that checks each candidate and discards leaking pieces early, and one based on PINI that certifies circuits too large to enumerate.
6) **A working compiler, seqc** (architecture in Sec. VIII, performance in Sec. X). Every security verdict about our circuits was confirmed by an independent verifier, with zero disagreements in more than 400,000 wire checks. A measured comparison with AGEMA and COMPRESS on twelve small programs is a supporting result, not the main claim: the compiler is first order, two shares and glitch-free, and seqc is behind both tools on one program.

Table II places this work next to the alternatives. Among the approaches we know of, it is the only one that saturates an e-graph of equivalent circuits and checks every result; search-based gadget optimisation [24] also explores rewritten gadgets under a security check, but by sampling rather than saturation. It is also the least mature, and Sec. XIII sets out its scope.

II. Background

A. Masking and gadgets

With two shares, a secret bit s is stored as a pair (s_0, s_1), where s_0 is a fresh random bit and $s_1 = s_0 \oplus s$. Each share on its own is a fair coin flip, and XOR-ing the two gives the secret back (Fig. 1).

XOR and NOT are easy to mask, since each can be applied to the shares separately. AND is harder, because multiplying shares mixes them together:

$$a \wedge b = a_0b_0 \oplus a_0b_1 \oplus a_1b_0 \oplus a_1b_1.$$

$a = a_0 \oplus a_1$

a	$r = a_0$	$a_1 = a \oplus r$	$a_0 \oplus a_1$
0	0	0	**0**
0	1	1	**0**
1	0	1	**1**
1	1	0	**1**

Fig. 1. Masking: the secret a is never stored, only two shares a_0 and $a_1 = a \oplus a_0$. Whatever a is, each share alone is a uniform coin flip, so one probe on a share learns nothing.

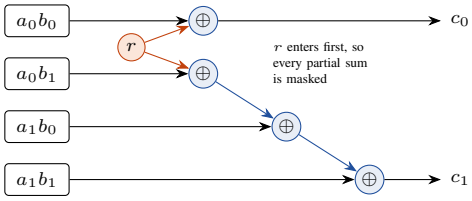


Fig. 2. The ISW masked AND: four share products and an XOR chain with one fresh random bit r, which enters first so that every partial sum is masked (4 AND + 4 XOR = 8 gates).

A masked AND therefore needs a *gadget*. The classic one is ISW [3], which adds one fresh random bit r (Fig. 2):

$$c_0 = a_0b_0 \oplus r,$$
$$c_1 = \big((r \oplus a_0b_1) \oplus a_1b_0\big) \oplus a_1b_1.$$

The order of the XORs matters: because r enters first, every partial sum is hidden by randomness. ISW costs 8 gates and one random bit, and its two outputs XOR to $a \wedge b$. DOM (domain-oriented masking) [4] and HPC1 (hardware private circuits) [5] compute the same product in different ways. A *refresh*, finally, gives a value fresh shares without changing it: $(x_0, x_1) \mapsto (x_0 \oplus r,\ x_1 \oplus r)$.

B. What counts as a leak

We consider an attacker who can probe one wire (this is called *first order*). A wire *leaks* if what the attacker sees on it depends on the secret, even just statistically (Fig. 3). In the simplest, *glitch-free* model, the attacker sees the wire's final value. Real wires, however, can flicker for a moment while signals arrive [7]. The *glitch-extended* model [6] captures this by letting the probe see everything that feeds into the wire. A *register*, which holds a value steady for one clock tick, blocks this flicker. Our checker decides security exactly, by trying every input and every random bit.

C. PINI

Enumerating every input is impossible for large circuits, and PINI [8] offers a way around it. A gadget is PINI (probe-isolating non-interference) if whatever a probe inside it reveals can be computed from input shares of a single index, the same index for every input, and shares of one index alone are noise. Two-share ISW is not PINI, because its product a_0b_1 needs share 0 of a and share 1 of b. PINI gadgets can be chained, and a circuit built only from them is secure at any size, so each gadget is checked once and the circuit as a whole never has to be enumerated.

TABLE II

HOW APPROACHES HANDLE MAKING A MASKED CIRCUIT CHEAPER WITHOUT MAKING IT LEAK (CELLS SUMMARISE EACH GROUP; TOOLS WITHIN A GROUP DIFFER; SUPERVER REWRITES EXPRESSIONS TO CHECK THEM, NOT TO OPTIMISE)

Approach	Examples	Cuts cost	Explores rewrites	Checks every result	Handles glitches	Large circuits	Tested on hardware
Cost-only optimiser	synthesis tools, plain e-graphs	**Yes**	**Yes**	**No**	**No**	**Yes**	**Yes**
Hand-designed gadgets	ISW, DOM, HPC; optimiser off	**No**	**No**	Partly	**Yes**	**Yes**	**Yes**
Verifiers only	maskVerif [19], VerifMSI, superVer	**No**	**No**	**Yes**	**Yes**	Partly	–
Masking compilers	AGEMA, COMPRESS, TETRIS	**Yes**	**No**	**Yes**	**Yes**	**Yes**	**Yes**
Search-based gadget optimisation	evolutionary search [24]	Partly	Partly	**Yes**	**Yes**	**No**	**No**
This work	security-constrained equality saturation	Partly	**Yes**	**Yes**	Partly	Partly	**No**

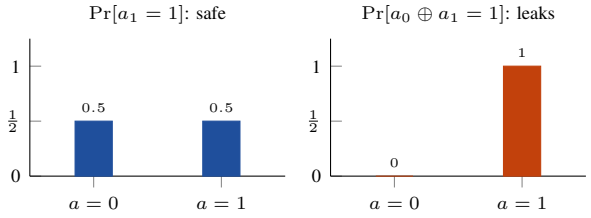


Fig. 3. What a leak looks like. Left: a single share is 1 with probability $\frac{1}{2}$ for both values of the secret (safe). Right: the two shares XORed together reveal the secret exactly (leaks). A wire is safe when $\Pr[w = 1 \mid a = 0] = \Pr[w = 1 \mid a = 1]$.

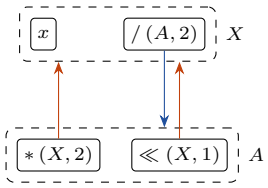


Fig. 4. An e-graph for $(x \cdot 2)/2$. Dashed boxes are e-classes (sets of equal expressions), solid boxes are e-nodes, and arrows point to operand classes. The cycle records that $(x \cdot 2)/2 = x$; nothing is deleted, both forms stay.

D. E-graphs and equality saturation

An e-graph groups expressions that are known to be equal into *e-classes* (Fig. 4). Equality saturation then works in two steps. It first applies every rewrite rule wherever it matches, until nothing new appears, so the order of the rules stops mattering. In practice we stop after a fixed number of rounds (six by default), so "every equivalent version" means every version reached within that bound. It then *extracts* a result, normally by picking the cheapest node in each class, and every use of a class becomes the same wire. One more property matters later: e-graph equalities must be a *congruence* (formally a partial congruence [9]), meaning that if $x = y$ then $f(x) = f(y)$ in every context. We use the egglog engine [2].

Key point. Saturation finds the options and extraction chooses one. That choice is made on cost alone, with no notion of security; Finding 1 shows what it picks.

III. FINDING 1: THE OPTIMISER BREAKS MASKING

We start from the ISW gadget and give the optimiser only ordinary, correct rules of Boolean algebra: commutativity, associativity, identities, distributivity and factoring. After saturation the e-graph holds 25 classes and 75 nodes, and the cheapest version has 7 gates instead of 8:

$$c_0 = a_0 b_0 \oplus r, \qquad c_1 = (r \oplus a_0 b_1) \oplus a_1 (b_0 \oplus b_1).$$

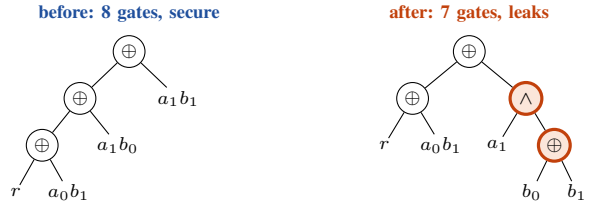
It still computes the right answer on all 32 inputs. But it contains a new wire, $b_0 \oplus b_1$, and since $b_1 = b \oplus b_0$,

$$b_0 \oplus b_1 = b_0 \oplus (b \oplus b_0) = b.$$

That wire *is* the secret b (Fig. 5). One probe reads it directly. We got the same circuit, with the same two leaking wires ($b_0 \oplus b_1 = b$ and $a_1(b_0 \oplus b_1) = a_1 \wedge b$), on two separate engines (our own e-graph and egglog), and the independent verifier VerifMSI confirms the leak.

Proposition 1 (Cost-only extraction is unsafe)**.** *There is a secure circuit and a set of correct Boolean rewrite rules for which the cheapest extracted circuit is insecure.*

The example above serves as the proof: no rule is wrong, and the leaking circuit wins only because it is cheaper.



$c_1 = \big((r \oplus a_0 b_1) \oplus a_1 b_0\big) \oplus a_1 b_1$ ISW, 8 gates, secure

$c_1 = (r \oplus a_0 b_1) \oplus a_1 (b_0 \oplus b_1)$ factored, 7 gates

$b_0 \oplus b_1 = b_0 \oplus (b \oplus b_0) = b$ the new wire is the secret

b	b_0	$b_1 = b \oplus b_0$	$b_0 \oplus b_1$
0	0	0	**0**
0	1	1	**0**
1	0	1	**1**
1	1	0	**1**

Fig. 5. The leak. Factoring $a_1 b_0 \oplus a_1 b_1 = a_1(b_0 \oplus b_1)$ takes the gadget from 8 gates to 7, but the new wire $b_0 \oplus b_1$ equals the secret b in every row of the truth table. The equations give the second output share before and after ($c_0 = a_0 b_0 \oplus r$ in both).

TABLE III
COST-ONLY VERSUS SECURE OPTIMUM FOR THE d-SHARE ISW AND (GATES, SHARED GATES COUNTED ONCE; FIRST ORDER, GLITCH-FREE)

Shares d	Plain ISW	Cost-only	Best secure found
2	8	7, **leaks**	8
3	21	16, **leaks**	17
4	40	27, **leaks**	28

Why the cheaper circuit leaks. A two-share ISW gadget turns $a \wedge b$ into four share products. Reordering and regrouping them never saves a gate; only factoring out a common term does. Two products share a term only if they share a share index, and factoring them then gives $a_i(b_0 \oplus b_1) = a_i \wedge b$, or the same with a and b swapped. So with two shares, **every gate-saving factoring puts a secret back together**.

A. *Is this only a two-share effect?*

With $d \geq 3$ shares the argument above weakens, because $b_0 \oplus b_1$ is a uniform bit rather than the secret. We therefore repeated the search for the d-share ISW gadget, enumerating every way to reorder the XOR terms and to factor products that share an index, and checking each wire exactly (Table III). We built the best secure circuit found and a cost-only circuit as netlists for each d (the search is per output, and gates shared between outputs are counted in the netlists) and checked every wire against VerifMSI: no disagreement on any of the 159 wires. The cost-only optimum still leaks for $d = 3$ and $d = 4$, because the last output share contains all d products with one index, and factoring them gives $a_{d-1} \wedge b$. The difference is that *secure* savings now exist, such as $a_2(b_0 \oplus b_1)$ for $d = 3$. So "every gate-saving factoring recombines a secret" is a two-share statement; the weaker statement "the cheapest circuit found by cost alone leaks" holds up to $d = 4$ in this model. This is a first-order, glitch-free measurement of the gadget only; it does not cover refreshes, DOM or HPC structure, or higher-order probing.

IV. FINDING 2: CHECKING SECURITY AT EXTRACTION FIXES IT

In the glitch-free model, whether a wire leaks depends only on the function it computes. That means we can label each e-class as safe or unsafe once, and reuse the label. The fix has three steps (Fig. 6):

1) label every e-class safe or unsafe;
2) pick the cheapest circuit, but never use an unsafe class as an internal wire;
3) check the result again with an independent tool.

The rules and the e-graph stay exactly the same; only the choice changes. The result is the secure 8-gate ISW gadget again (Table IV). **Security costs one gate.**

Why not simply ban the factoring rule? Banning it would not help, for three reasons. Factoring is useful elsewhere (later it takes a circuit from 20 to 11 and stays secure). Other moves leak too, such as moving r to the end in ISW or dropping a needed refresh. And a leak depends on which wires end up in the circuit, not on which rule produced them. So we let every rule fire and check the result.

TABLE IV
WIRE-LEVEL RESULTS ON ISW (COST: GATES)

Rule set	Classes	Gates	Security
Core (commute, associate, identities)	23	8	secure
Full (+ distributivity, factoring), cost only	25	7	**leaks**
Full, constrained extraction	25	8	secure

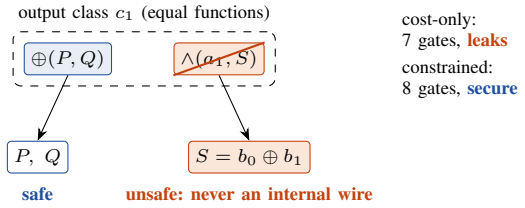


Fig. 6. Security-constrained extraction. Same rules and same e-graph; every class is labelled safe or unsafe, and the unsafe class $S = b_0 \oplus b_1$ is never used as an internal wire. Ignoring security gives 7 gates and leaks; enforcing it gives 8 gates and is secure.

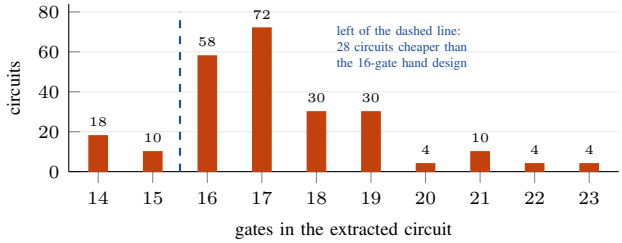


Fig. 7. Chain scale. Gate counts of the 240 sampled extractions of a two-gadget ISW chain: none is secure, and the 28 that beat the 16-gate hand design all leak. VerifMSI agrees on all 6,054 wires.

Is there a cheaper secure circuit somewhere? We tried a larger example: two ISW gadgets in a row, computing $(a \wedge b) \wedge d$, which takes 16 gates by hand and is secure. We took egglog's 40 cheapest extractions of the second output and the 6 distinct ones it found for the first, and paired them in all $6 \times 40 = 240$ ways. None of the 240 is secure, including the 28 that are cheaper than the hand design (Fig. 7). This is a sample, not a proof, and it suggests that ordinary algebra alone offers no free savings here. Finding 3 shows that the moves designers use to save cost are not available at this level.

V. FINDING 3: DESIGNERS' TRICKS DO NOT FIT A STANDARD E-GRAPH

Masking designers save cost with a different set of moves: refreshes, gadget swaps and registers (Fig. 8). Refreshes and gadget swaps keep the *secret* the same and change the *shares*; we call this *re-sharing*. A register keeps both, but changes what a probe can see under glitches.

Move	What it does
Refresh	$(x_0, x_1) \mapsto (x_0 \oplus r,\ x_1 \oplus r)$: same secret, fresh shares
Gadget swap	$\mathrm{ISW}(a, b) \leftrightarrow \mathrm{DOM}(a, b)$: same product, different wires
Register	holds a value steady; under glitches it cuts what a probe can see, much as a refresh would
Random-bit reuse	one random bit shared between gadgets (not modelled here)

Fig. 8. Re-sharing: the moves masking designers use. Refreshes and gadget swaps keep the secret and change the shares; a register keeps both but changes what a glitch-extended probe sees; random-bit reuse is not modelled.

$$x = y \ \Rightarrow \ f(x) = f(y)$$

x	y	r	x_0	$y_0 = x_0 \oplus r$	x_1	$y_1 = x_1 \oplus r$
1	1	0	1	1	0	0
1	1	0	0	0	1	1
0	0	1	0	**1**	0	1
1	1	1	0	**1**	1	0

Fig. 9. Why re-sharing is not a congruence. x and $y = \mathsf{refresh}(x)$ always hold the same secret (columns x and y), but share 0 differs in rows 3 and 4, so the context "read share 0" gives $f(x) = 0$ but $f(y) = 1$ in those rows.

Re-sharing is not a congruence. Let y be a refresh of x, so $y_0 = x_0 \oplus r$ and $y_1 = x_1 \oplus r$. Both hold the same secret. Now look only at share 0: whenever $r = 1$, x_0 and y_0 differ. So x and y are equal as secrets but not as wires. That breaks the congruence rule, and **a correct wire-level e-graph is never allowed to treat them as equal** (Fig. 9).

A wire-level optimiser therefore cannot *see* the designers' moves at all. We found no earlier statement of this point for e-graphs; it applies the known fact that e-graph equalities form a congruence [9] to masked circuits (search record in Appendix B).

A. A second e-graph, over secret values

Our answer is to move the e-graph up a level, so that each class stands for a secret value rather than a particular set of wires (Fig. 10). A term is built from inputs, XOR, NOT, refresh, registers and gadgets:

$$x ::= \mathsf{in}(s) \mid \mathsf{xor}(x, x) \mid \mathsf{not}(x) \mid \mathsf{refresh}(x) \mid \mathsf{reg}(x) \mid g(x, x),$$

where g is one of isw, dom or hpc1. Its value is the obvious one: $\mathsf{V}(g(x, y)) = \mathsf{V}(x) \wedge \mathsf{V}(y)$, and refresh and registers do not change the value. Three groups of rules act on it: **R1** is ordinary Boolean algebra on values (including factoring), **R2** says the three gadgets are interchangeable, and **R3** says a refresh can be removed. To measure cost, we turn a term back into a two-share circuit (*lowering*), with one fresh random bit for every ISW or DOM gadget and every refresh (HPC1, which includes a refresh, uses two), and count gates plus random bits.

Proposition 2 (The value-level rules are safe to use)**.** *If* $\mathsf{V}(x) = \mathsf{V}(x')$ *and* $\mathsf{V}(y) = \mathsf{V}(y')$*, then* $\mathsf{V}(\mathsf{op}(x, y)) = \mathsf{V}(\mathsf{op}(x', y'))$*, so R1–R3 are valid e-graph rules. Lowering is also correct: the output shares of a lowered term always XOR to its value.*

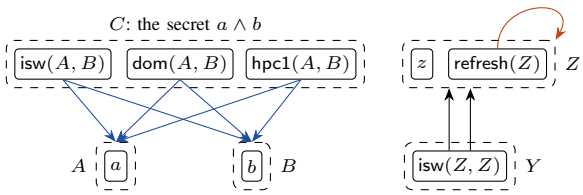


Fig. 10. The value-level e-graph. A class stands for a secret value: three gadgets build the same value $a \wedge b$ (R2), and a refresh joins its own class as a self-loop (R3).

Proof. The value of a term is defined piece by piece from the values of its parts, and lowering is correct by induction (a refresh is correct because r cancels out, and each gadget by its own correctness). □

Security is no longer part of the equality at all, so it has to be enforced when the circuit is chosen. At this level a single safe or unsafe label per class no longer works (Proposition 3).

VI. Finding 4: One Label Per Class Is No Longer Enough

Proposition 3 (Safety is not a property of the e-class)**.** *With refresh removal (R3),* $\mathsf{isw}(z, z)$ *and* $\mathsf{isw}(z, \mathsf{refresh}(z))$ *end up in the same e-class, yet the first leaks and the second is secure. With R3 as the only rule, picking one node per class always returns* $\mathsf{isw}(z, z)$*, while picking whole terms can return the secure one.*

In $\mathsf{isw}(z, z)$, both inputs are the same pair of shares, so one product wire is 1 half the time when $z = 0$ and never when $z = 1$; it leaks z. With a refresh on one input, every product wire is 1 a quarter of the time, whatever z is (Fig. 11). Because R3 makes the refresh a loop back into z's own class, the usual one-node-per-class extraction cannot use it, so both inputs become the same wire (full proof in Appendix A). This holds for refresh removal alone. Recounting every acyclic one-node-per-class choice for this example shows how quickly it fails once more rules are present. With refresh removal alone there is one choice, and it leaks. With distributivity (R1) added there are 338 choices and 16 are secure, for instance $\mathsf{isw}(a, a \oplus b) \oplus \mathsf{isw}(b, a \oplus b)$. With gadget swaps (R2) added instead, the HPC1 gadget, which refreshes its second operand itself, is also secure, and with all three rule groups 2,226 of 8,502 choices are secure. The claim is therefore not that one-node-per-class extraction can never find a secure circuit, only that it can miss every one. At the wire level, a single label per class still works.

A. Safe method 1: check each candidate, cut leaks early

Instead of choosing one node per class, we choose whole *terms*, so the same class can be built differently in different places. Each class keeps its $k = 8$ cheapest terms. On the value-level fixtures the results were identical for $k = 2$, 8 and 32. The $2{\times}2$ multiplier behaves differently: its secure optimum is 69 for k from 2 to 8 and 75 for $k = 16$ and 32. Extraction is therefore a search, and the costs we report are the best found, not proven optima. The extractor ranks child terms by their

$\mathsf{isw}(z, z)$: **leaks**

z	z_0	z_1	$z_0 z_1$
0	0	0	0
0	1	1	**1**
1	0	1	0
1	1	0	0

$\mathsf{isw}(z, \mathsf{refresh}(z))$: **secure**

z	z_0	r	z_1	$z_1' = z_1 \oplus r$	$z_0 z_1'$
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	1	**1**
0	1	1	1	0	0
1	0	0	1	1	0
1	0	1	1	0	0
1	1	0	0	0	0
1	1	1	0	1	**1**

Fig. 11. Why one safe-or-unsafe label per class fails. Left, $\mathsf{isw}(z, z)$: both operands are one sharing, and the product wire $z_0 z_1$ is 1 in one of two cases when $z = 0$ and never when $z = 1$, so it leaks z. Right, $\mathsf{isw}(z, \mathsf{refresh}(z))$: the product is 1 in one of four cases for either value of z, so it is safe. Refresh removal puts both terms in one e-class.

standalone cost, while the cost of a term depends on what it shares, so a wider frontier can push out a term that is cheap in context. Each candidate is lowered to wires, and every wire is checked exactly.

Proposition 4 (Leak cut)**.** *In both models, if a piece of a circuit has a leaking wire, then every larger circuit containing that piece also leaks.*

So a leaking piece can be thrown away as soon as it is found, and every result is secure by construction. On the squaring example, $\mathsf{isw}(z, z)$ (cost 11) is cut and $\mathsf{isw}(z, \mathsf{refresh}(z))$ (cost 14) is kept. The price is speed: 0.3 s on a chain of depth 3, 5.4 s at depth 4 and 133 s at depth 5.

B. Safe method 2: PINI brings back one label per class

We call a class *composable* if it contains a PINI gadget whose inputs are also composable. We checked each operator exhaustively and with VerifMSI: XOR, NOT, refresh and HPC1 are PINI, while ISW and DOM are not. Inside composable classes a single safe label works again, and the usual one-node-per-class extraction is safe, **without checking any inputs at all**. This is what lets the method scale. It is the same composition property that COMPRESS and TETRIS rely on to choose gadgets without enumerating inputs; what we believe is new here is using it inside extraction from an e-graph.

VII. Results

A. Savings appear only where there is waste

Fig. 12 shows the cheapest secure circuit found from several starting points. Every saving comes from one of three kinds of waste: a refresh that was not needed, two products that can be factored, or a gadget that can be swapped for a cheaper one. Circuits that are already tight do not move. VerifMSI confirms every secure result (27 circuits, 914 wires). The security check still matters at this level: if refresh removal is allowed, a cost-only optimiser returns $\mathsf{isw}(z, z)$ (cost 11, leaks) instead of $\mathsf{isw}(z, \mathsf{refresh}(z))$ (cost 14, secure).

Table V switches the three rule groups on and off. Each saving has one source: refresh removal (R3) gives the over-refreshed chain, Boolean algebra with factoring (R1) gives the shared-factor program, and gadget swaps (R2) give the HPC1 chain. No other combination saves anything, and the only configurations whose cost-only optimum leaks are those that contain R3.

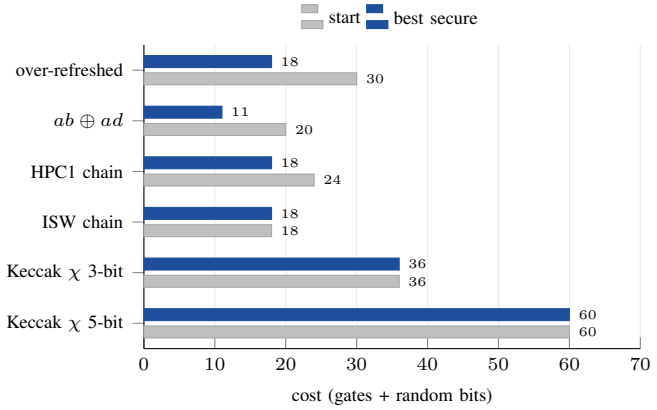


Fig. 12. Value-level results (cost = gates + random bits). Refresh removal, factoring and gadget swaps save cost where the start circuit had slack: over-refreshed chain 30 to 18 (R3), two ANDs sharing a factor 20 to 11 (R1), HPC1 chain 24 to 18 (R2, glitch-free only). The hand-written chain and Keccak χ are tight and do not improve.

TABLE V
Rule Ablation: Cheapest Secure Cost (Gates + Random Bits). Bold Marks a Saving Over the Start; † Marks a Cost-Only Optimum of 11 That Leaks

			one rule group			two rule groups			
Circuit	Start	none	R1	R2	R3	R1+R2	R1+R3	R2+R3	all
ISW chain	18	18	18	18	18	18	18	18	18
over-refreshed	30	30	30	30	**18**	30	**18**	**18**	**18**
$z \wedge z$	14	14	14	14	14†	14	14†	14†	14†
$ab \oplus ad$	20	20	**11**	20	20	**11**	**11**	20	**11**
HPC1 chain	24	24	24	**18**	24	**18**	24	**18**	**18**
Keccak χ, 5-bit	60	60	60	60	60	60	60	60	60

B. Finding 5: glitches and large circuits

Glitches. A glitch is a brief wrong value on a wire while signals are still arriving (Fig. 13). A register blocks it, so for the attacker a register has an effect similar to a refresh. To a cost-only optimiser, however, a register looks like a wire that does nothing, so it deletes it (Table VI). Our check puts the registers back, and on the unregistered ISW gadget it adds

$w = (a_0 \oplus r) \oplus a_1$, with $a = 1$, $a_0 = 0$, $a_1 = 1$, $r = 1$

Time	Event	$u = a_0 \oplus r$	$w = u \oplus a_1$	Wire holds
t_0	nothing has arrived	0	0	nothing useful
t_1	a_0, a_1 arrive; r still 0	0	**1**	**the secret a**
t_2	$r = 1$ arrives	1	0	$a \oplus r$, random

Fig. 13. A glitch. While signals arrive, the wire $w = (a_0 \oplus r) \oplus a_1$ briefly holds the secret a before r arrives; its final value passes a basic check. A probe that sees the wire's history sees a, and a register stops this.

TABLE VI
REGISTERS UNDER GLITCHES (COST: GATES + REGISTERS)

Circuit	Start	Registers ignored	Constrained
Registered DOM AND	10	5, **leaks**	10, secure
DOM chain (registered)	20	10, **leaks**	20, secure
ISW AND, no registers	8	7, **leaks**	10, secure
DOM AND, no registers	8	5, **leaks**	10, secure

TABLE VII
EXACT CHECKING VERSUS PINI COMPOSITION (ASSIGN.: TWO BITS PER SECRET PLUS THE FRESH RANDOM BITS; VERIFMSI COLUMN: ITS SYMBOLIC CERTIFICATE)

Circuit	Assign.	Exact	PINI	Time	VerifMSI
Keccak χ, 6-bit	2^{24}	secure	secure	0.01 s	103/103
Keccak χ, 8-bit	2^{32}	too large	secure	0.01 s	137/137
Keccak χ, 32-bit	2^{128}	too large	secure	0.05 s	545/545
Chain, 15 gadgets (HPC1)	2^{62}	too large	secure	0.01 s	212/212

two registers exactly where they are needed. Of egglog's own 784 cheapest candidates for a DOM chain, none is glitch-secure. Our glitch checker agrees with VerifMSI on all 167 probes of the ten distinct circuits behind Table VI, and on all 21,869 probes of the 784 candidates, whose end-to-end verdicts VerifMSI also confirms.

Large circuits. Here Keccak χ means the map $x_i \oplus (\neg x_{i+1} \wedge x_{i+2})$ on a cyclic row of n bits. Keccak itself uses $n = 5$ (the 5-bit row of Fig. 12), and wider rows are a scaling benchmark of the same circuit shape, not a Keccak state size. The count 2^{128} for $n = 32$ is two bits for each of the 32 secrets (its two shares) plus 64 fresh random bits, an upper bound on what an exhaustive check would enumerate, not the number of unmasked states. Checking every input stops at around 2^{26} combinations. PINI-based extraction does not need to enumerate anything (Table VII), and none of 234 randomly built PINI circuits leaks. The price is size: an HPC1 gadget costs 12 where ISW costs 9, so PINI results are up to 33% larger (25% on the Keccak rows, 33% on the chains, less on programs with savings). This result holds in the glitch-free model.

VIII. SYSTEM ARCHITECTURE

Fig. 14 shows how the tool is organised, in four layers. The *front end* reads a program and masks it. The *optimiser* holds the two e-graphs and the two ways of choosing a circuit. The *oracles* are the security checkers that the optimiser consults.

TABLE VIII
COMPONENTS OF THE TOOL

Component	Role
Parser	Reads the source language; one syntax tree per output.
Masking pass	Two shares per value; ISW per AND; refresh policy keeps operand sharings independent.
Wire-level e-graph	Boolean rewriting of masked wires (RQ1, RQ2); our own engine and an egglog port.
Value-level e-graph	One class per secret value; R1–R3 as rules (RQ3).
Per-term extraction	k cheapest terms per class, lowered and checked; leaking terms cut for good.
PINI extraction	Class-level extraction on composable classes; no enumeration (RQ4).
Labeller	Bit-parallel safe/unsafe labels; 0.4 s for a depth-5 chain against an estimated 61 min for the reference labeller.
Exact checker	Ground truth for first-order security, glitch-free and glitch-extended.
VerifMSI	External oracle; any disagreement stops the work until fixed.
Emitter	Netlist or Verilog with `a_s0`, `a_s1`, … ports.

TABLE IX
COMPILED PROGRAMS (COST: GATES + RANDOM BITS)

Program	Start	Secure	Saving	Cost-only
$a \wedge b \oplus a \wedge c$	26	11	58%	11, secure
$z \wedge z$, $z = a \oplus b$	14	14	0%	11, **leaks**
multiplexer	27	13	52%	13, secure
full adder	30	24	20%	24, secure
majority, 2 AND	28	22	21%	22, secure
majority, OR form	65	41	37%	41, secure

The *back end* checks the result and writes it out, and a test harness reruns everything. Table VIII describes each part in one line.

IX. THE SEQC COMPILER

Fig. 15 follows a program through the compiler. The front end reads a small language with `input`, `out` and the operators `~`, `&`, `^` and `|`. The masking pass turns each AND into an ISW gadget and refreshes its second input (a refreshed operand that is used twice is refreshed once). egglog then saturates the e-graph with rules R1–R3. Extraction runs twice, once ignoring security and once enforcing it, so the cost of security is always visible. The result is checked (same output as the source on every input, no leaking wire, VerifMSI agrees) and written out as a netlist or Verilog. The Verilog is marked as secure in the glitch-free model only.

Table IX shows the compiled programs. Every result matches its source on all inputs, has no leaking wire, and is confirmed by VerifMSI. On $z \wedge z$ the optimiser is tempted to drop a refresh that is needed, and seqc refuses, so the main finding shows up on a real compiled program too. The largest program of this table, a majority vote written with ORs (five gadgets as written, four after optimisation), compiles from 65 to 41 in 83 s.

Table X adds six larger programs, including the 5-bit S-box of Ascon and the 4-bit S-box of PRESENT. Each program

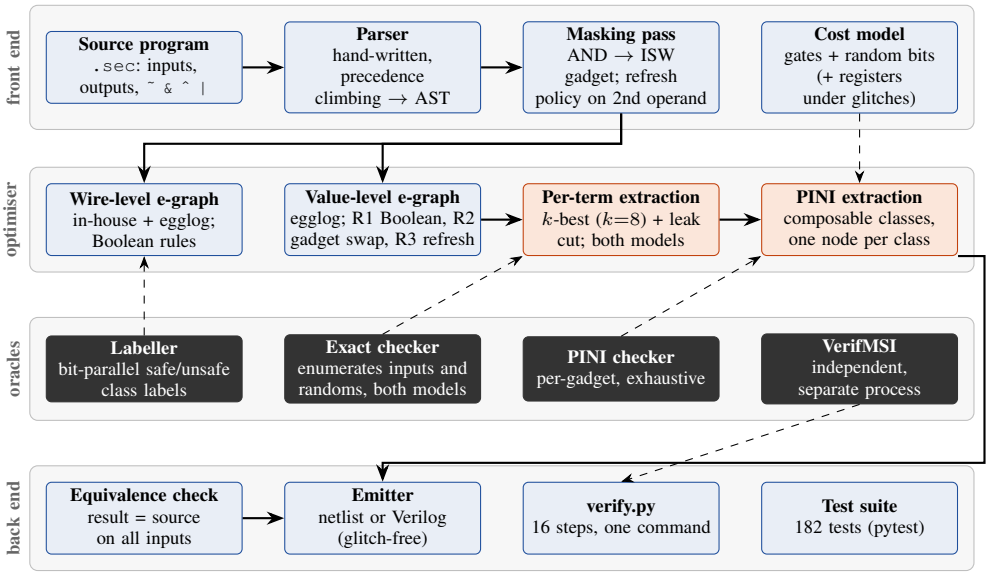


Fig. 14. Architecture of the tool. Solid arrows are the compilation flow; dashed arrows show which oracle each stage consults. Extraction (orange) is the only optimisation stage that depends on security; VerifMSI is never linked into our code and is used only to cross-check.

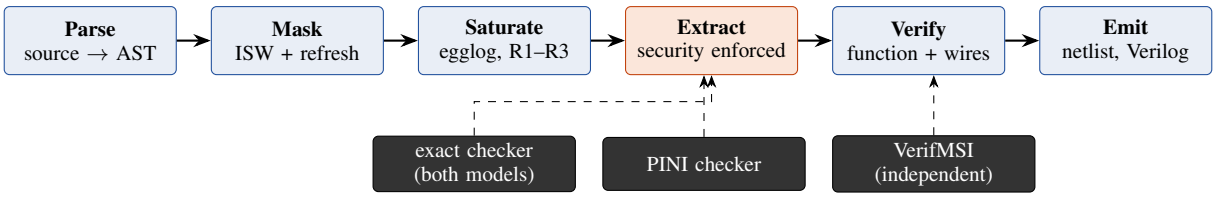


Fig. 15. The seqc pipeline. Extraction (orange) is the only optimisation stage that consults the security checkers; VerifMSI runs as a separate process.

TABLE X
LARGER COMPILED PROGRAMS (COST: GATES + RANDOM BITS; FIRST ORDER, TWO SHARES, GLITCH-FREE)

Program	Start	Cost-only	Secure	Saving	Rand.	Time
2-bit comparator	58	48, secure	48	17%	8→4	27 s
2-bit adder	60	48, secure	48	20%	8→4	75 s
2×2 multiplier	70	66, **leaks**	69	1%	10→8	6 s
4:1 multiplexer†	48	39, secure	39	19%	6→3	8 s
Ascon S-box†	88	78, secure	78	11%	10→5	14 s
PRESENT S-box	134	101, secure	101	25%	12→8	132 s

†Run with frontier width $k = 2$ and 4 saturation rounds; the default $k = 8$ with 6 rounds did not finish within 40 minutes. With 4 rounds, $k = 2$, 4 and 8 give the same cost on both programs.

was first checked against its specification (the published S-box tables, or integer arithmetic) on every input, and each compiled result matches its source, has no leaking wire, and agrees with VerifMSI on all 443 wires. Savings are between 11% and 25% except on the multiplier, where the cost-only optimum is again cheaper and leaks, and the secure circuit saves only 1%. The emitted Verilog of all twelve programs also passes an Icarus simulation: every input value is driven with eight random sharings, and the two output shares must XOR to the value the source computes. A copy of the adder with one XOR turned into an AND fails it. This checks function, not security. All twelve programs, these six included, are compared with AGEMA and COMPRESS in Table XI.

X. PERFORMANCE COMPARISON

We compare seqc with four baselines, run on the programs of the tables below (the hand-written design only where one exists): a *cost-only optimiser* (the same e-graph and rules with ordinary extraction, which is what standard equality-saturation tools do), the *hand-written* masked design, and two published masking tools, AGEMA [11] and COMPRESS [17].

A. Security: ours versus a cost-only optimiser

Fig. 16 lists every case of the gadget, chain and glitch experiments where the cost-only optimiser and our tool disagree. Among the compiled programs, $z \wedge z$ is in it and the multiplier of Table X is a second case, where the cost-only result (66) leaks and the secure one costs 69. Each time, the cost-only result is cheaper and leaks, and ours is secure. The gap is the price of security: one gate on ISW, three on squaring, three on the unregistered ISW gadget (one gate and two registers), two on the two-gadget ISW chain, and the full set of registers on DOM circuits. In addition, none of egglog's 784 cheapest DOM-chain candidates is glitch-secure, and none of the 240 sampled chain circuits is secure.

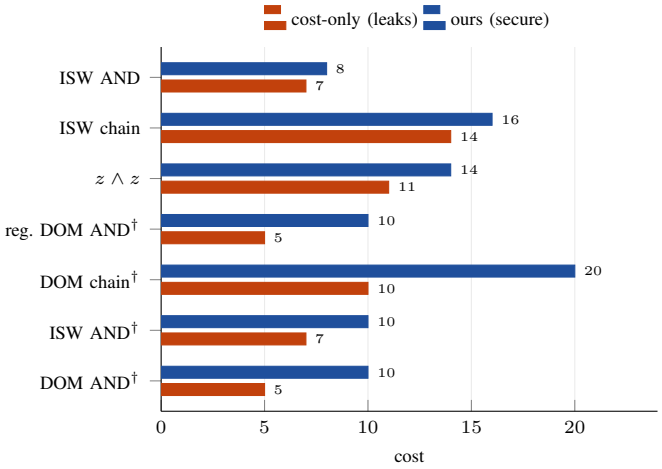


Fig. 16. Price of security. Every circuit of the gadget, chain and glitch experiments on which the cost-only optimiser and our tool disagree: the cost-only result always leaks. Costs are gates (ISW AND, chain), gates + random bits ($z \wedge z$), or gates + registers (†, glitch-extended model).

B. Cost: ours versus the hand-written design

Where the starting circuit has waste, the secure result is 11–58% cheaper: 58% on $a \wedge b \oplus a \wedge c$ from the compiler's start of 26, 52% on the multiplexer, 45% on the same logic from its hand-written start of 20, 40% on the over-refreshed chain, 37% on the OR-form majority, 25% on the HPC1 chain, 21% on majority and 20% on the full adder, and the larger programs of Table X save 11–25% apart from the multiplier (Fig. 12, Table IX). On tight designs (the hand-written ISW chain and Keccak χ) it finds nothing to save, and in the glitch-free model it returned no leaking circuit on any program we ran, by construction of the leak cut. The PINI route is the one case that costs more, up to 33% above the ISW-based design, in return for certifying circuits too large to check directly.

C. Head-to-head with AGEMA and COMPRESS

We ran both tools on the twelve programs of Tables IX and X with two shares, all on one machine (an AMD Ryzen 5 5500U laptop, 12 threads, 5.6 GB, Linux; Yosys 0.70, GHDL 7.0, COMPRESS commit 26ccd9c, AGEMA commit 3de80c1). COMPRESS was run with its default gadget library at latencies 2 to 4; the multiplier and the OR-form majority have no schedule at latency 2, and every other design it produced passed its own behavioural simulation. Its area is the area of the synthesised design from Yosys with the Nangate45 library, the flow its authors use. AGEMA was run through its Yosys front end with its HPC2 and HPC3 gadgets. Its gadgets are VHDL, which we read with the GHDL plugin of Yosys, so AGEMA's area is measured on the same library, not estimated. All areas are in gate equivalents (GE, 1 GE $= 0.798\,\mu\text{m}^2$). Table XI shows the results. As a check, an earlier COMPRESS run on the six programs of Table IX gave the same gadget counts, random bits and area (for example 148.0 GE on $a \wedge b \oplus a \wedge c$), so our flow reproduces it.

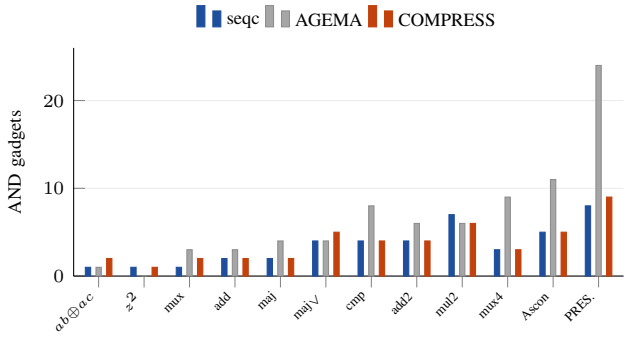


Fig. 17. Masked AND gadgets needed per program (fewer is better). Each gadget costs area and fresh randomness in every masking scheme.

seqc needs no more gadgets than either tool on ten of the twelve programs. It needs strictly fewer gadgets than AGEMA on eight programs (tying on two) and strictly fewer than COMPRESS on four (tying on seven). AGEMA's front end optimises the logic for *unmasked* area, so it turns the multiplexers and the S-boxes into NAND and NOR networks, each gate of which then becomes a masked gadget (24 gadgets for the PRESENT S-box against seqc's 8). COMPRESS keeps the logic exactly as written, so it pays for every AND in the source. The two exceptions are $z \wedge z$, where AGEMA's synthesis applies $x \wedge x = x$ and removes the gadget (a rule we deliberately left out to study the refresh), and the multiplier, where seqc's secure optimum has 7 gadgets and both tools have 6 (its cost still falls from 70 to 69, because random bits fall from 10 to 8 while one gadget is added). **The savings carry over into a glitch-robust flow on some programs, not all**: feeding seqc's chosen logic into COMPRESS makes its design smaller on four programs (by 45% on $a \wedge b \oplus a \wedge c$, 27% on the multiplexer, 6% on the Ascon S-box and 11% on the PRESENT S-box), the same on five, and larger on three (by 12% on the multiplier, 3% on the OR-form majority, where the random bits still drop from 9 to 6, and 1% on the comparator). seqc's own area is below COMPRESS's on every program and below AGEMA's on all but $z \wedge z$, where AGEMA removes the gadget, but this is not a fair comparison: it has no registers, no random-number generator and only glitch-free security. Fig. 17 summarises the gadget counts.

On speed, all on the machine above, AGEMA takes 8–17 ms per run and COMPRESS's solver 0.04–0.68 s over all latencies tried. seqc takes at most 0.2 s on the five smallest programs and between 6 s and 132 s on the larger ones, where its exact checking is the limit (83 s for the OR-form majority, 75 s for the 2-bit adder, 132 s for the PRESENT S-box).

D. Do the savings need equality saturation?

A fair objection is that any logic minimiser might find the same savings. Table XII compares seqc with a plain minimiser, Yosys with ABC mapping to AND and XOR gates, on twelve programs (the six above and the six of Table X). The minimiser knows nothing about masking. It needs more AND gates than seqc on eight programs, the same number on two, and fewer

TABLE XI
MEASURED HEAD-TO-HEAD ON TWELVE PROGRAMS (TWO SHARES, ONE MACHINE). AREA IN GE, NANGATE45. "GADGETS" COUNTS MASKED AND-TYPE GADGETS; "RND" COUNTS FRESH RANDOM BITS

	seqc (this work)		AGEMA		COMPRESS		
Program	gadgets / rnd	area[a]	gadgets / rnd[b]	area[c]	gadgets / rnd	area	area on seqc's logic[d]
$a \wedge b \oplus a \wedge c$	1 / 1	12.0	1 / 1	73.0	2 / 2	148.0	**81.7** (-45%)
$z \wedge z$, $z = a \oplus b$	1 / 2	16.7	0 / 0	4.0	1 / 1	70.3	70.3 (same)
multiplexer	1 / 1	16.0	3 / 3	207.0	2 / 2	148.7	**108.3** (-27%)
full adder	2 / 2	32.0	3 / 3	237.3	2 / 2	167.3	167.3 (same)
majority, 2 AND	2 / 2	22.0	4 / 4	285.0	2 / 2	152.0	152.0 (same)
majority, OR form	4 / 4	59.7	4 / 4	285.0	5 / 9	282.3	290.7 (+3%), rnd 9 $\rightarrow$ 6
2-bit comparator	4 / 4	66.3	8 / 8	517.7	4 / 5	275.0	279.0 (+1%)
2-bit adder	4 / 4	73.0	6 / 6	543.7	4 / 6	302.0	302.0 (same)
2×2 multiplier	7 / 8	58.7	6 / 6	421.7	6 / 10	374.7	420.0 (+12%)
4:1 multiplexer†	3 / 3	53.3	9 / 9	641.3	3 / 5	213.0	213.0 (same)
Ascon S-box†	5 / 5	98.0	11 / 11	937.0	5 / 5	440.3	**411.7** (-6%)
PRESENT S-box	8 / 8	133.7	24 / 24	1753.7	9 / 11	627.0	**555.7** (-11%)

[a] Glitch-free, combinational, no registers and no random-number generator; not directly comparable with the glitch-robust, pipelined designs of AGEMA and COMPRESS. [b] HPC2 (one random bit per gadget). [c] Measured: best of HPC2 and HPC3, AGEMA's VHDL gadgets and pipeline registers synthesised together. Neither tool's area includes a random-number generator. [d] COMPRESS run on the Boolean structure seqc chose, each gadget as an AND; COMPRESS keeps its own glitch-robust guarantee, and the first number is its best-latency area. Changes are against the COMPRESS area in the previous column. † seqc with $k = 2$.

TABLE XII
AND GADGETS WITHOUT AN E-GRAPH (YOSYS/ABC) VERSUS SEQC, AND SEQC AREA

Program	Written	ABC	seqc	Rnd.	GE
$a \wedge b \oplus a \wedge c$	2	1	1	1	12.0
$z \wedge z$	1	0	1	2	16.7
multiplexer	2	3	1	1	16.0
full adder	2	3	2	2	32.0
majority, 2 AND	2	4	2	2	22.0
majority, OR form	5	4	4	4	59.7
2-bit comparator	4	8	4	4	66.3
2-bit adder	4	6	4	4	73.0
2×2 multiplier	6	6	7	8	58.7
4:1 multiplexer†	3	9	3	3	53.3
Ascon S-box†	5	13	5	5	98.0
PRESENT S-box	9	26	8	8	133.7

Written: ANDs in the source (shared terms once). ABC: ANDs after Yosys/ABC. seqc: gadgets and random bits of the secure optimum. † $k = 2$.

on two: $z \wedge z$, where it applies $x \wedge x = x$, and the multiplier. On eight programs it even needs more ANDs than the source already had, since ABC maps for area, not for the number of ANDs. This is a weak baseline, and a minimiser built to reduce the AND count would do better: for the PRESENT S-box it is known exactly that the minimum number of ANDs is 4 [25], against 8 gadgets for seqc and 9 as written. We did not run such a minimiser, and a circuit with the fewest ANDs is not by itself secure, but on this evidence seqc is not optimal in gadget count there. Table XII also gives the area of seqc's own Verilog (Yosys 0.70, Nangate45, 1 GE $= 0.798\,\mu\text{m}^2$, no registers, glitch-free model only).

E. Speed and scale

Table XIII gives the measured run times.

TABLE XIII
MEASURED SPEED AND SCALE

Task	Result
Per-term extraction, ISW chain depth 3 / 4 / 5	0.3 s / 5.4 s / 133 s
Labeller, depth-5 chain: bit-parallel / reference	0.4 s / ≈61 min (est.)
PINI extraction, Keccak χ 32-bit (2^{128} inputs)	0.05 s, 545/545 wires
PINI extraction, chain of 15 gadgets, as HPC1 (2^{62} inputs)	0.01 s, 212/212 wires
Exact checker, largest input space	$\approx 2^{26}$ points
seqc, OR-form majority (5 gadgets written, 4 after), 65 $\rightarrow$ 41	83 s
Full verification, all 16 steps, clean checkout	≈9 min (533 s)

XI. VERIFICATION

We never relied on our own checker alone. Every security verdict about our circuits was cross-checked by VerifMSI [12], a published verifier we did not write, run as a separate program (Table XIV). One command, `findings/verify.py --full`, reruns the core experiments in 16 steps, and all of them pass on a clean checkout, along with 182 automated tests; the head-to-head, the larger programs and the extra cross-checks have their own scripts in `experiments/bench_extra`. Agreement on these circuits is strong evidence, but it is not a proof for every possible circuit. We also tested the verifier itself, and use its exact check, rather than its one-sided symbolic shortcut, as the reference.

XII. RELATED WORK

Masking compilers. AGEMA [11] turns an unprotected netlist into a masked one using PINI gadgets. It may simplify the *unmasked* logic first, where rewrites cannot hurt security. COMPRESS [17] takes a Boolean circuit as given and, with a constraint solver, chooses the gadgets, schedules the pipeline registers and removes logic duplicated between gadgets; TETRIS [18] likewise takes the circuit as given and assigns PINI gadgets under a randomness or latency budget with fast heuristics. Tornado [10] adds refreshes wherever its verifier finds an attack. Feldtkeller et al. [16] let gadgets

TABLE XIV
AGREEMENT WITH VERIFMSI

Checked	Circuits / wires	Disagree
Exact checker	1,216 / 21,760	0
Bit-parallel labeller	1,265 / 22,463	0
Value-level circuits	27 / 914	0
Sampled chain extractions	240 / 6,054	0
Random PINI-built circuits	234 / 4,608	0
Compiled runs, Table IX	7 / 178	0
Larger compiled programs	6 / 443	0
Two- to four-share ISW	6 / 159	0
Extractions of $z \wedge z$	8,844 / 348,973	0
Emitted Verilog files read back	12 / 603	0
Total wire checks	**406,155**	**0**
Glitch checker, ten circuits (probes)	167	0
Glitch frontier, 784 circuits (probes)	21,869	0
PINI verdicts (gadgets)	6	0

share random bits, which we do not model. Apart from the search-based work below, none of them explores equivalent *masked* circuits by rewriting. The type-directed compiler of Barthe et al. [20] also inserts refreshes automatically, but for a fixed circuit and with a stronger notion (strong non-interference, SNI) than the probing notion used here. Kumar et al. [23] lower the fresh randomness of first-order masked circuits by graph colouring, and, in their words, only rewire the randomness inputs and leave the logic unchanged.

Verifiers. VerifMSI [12] is our independent referee; maskVerif [19] and SILVER [21] are other well-known checkers. superVer [15] uses e-graphs to *check* masked circuits rather than optimise them. The eVer verifier [13] proves joint independence for several masking schemes; its authors also use it to optimise away a superfluous sequence of refresh applications and to optimise refresh gadgets for polynomial and inner-product masking at two shares, re-verifying each result. We remove redundant refreshes automatically, inside an optimiser, on whole programs.

Optimisers that break masking. MaskedHLS [14] and a study for the German BSI [28] report that standard synthesis steps, such as re-timing, can weaken masking. MaskedHLS responds by placing registers where glitch-robust gadgets require them and balancing the paths. We explain why this happens in equality saturation, and show how to fix it.

E-graphs for hardware. E-Syn [22] applies equality saturation to gate-level Boolean logic with technology-aware costs and reports area and delay savings over ABC; its abstract mentions no security property, so it is the kind of optimiser whose output Finding 1 warns about.

Search-based optimisation of masked gadgets. Koblah et al. [24] optimise individual masked gadgets (the COMAR gadgets, which reuse fresh masks, and the HPC1–HPC3 hardware-private-circuit families, first order) at the standard-cell level with an evolutionary search: a genetic algorithm over a graph of NanGate45 gates, with sensitivity-guided crossover and four kinds of mutation. Candidates are ranked by a surrogate for power and area, functional equivalence is checked by SAT, and the final candidates are verified for PINI security with the VERICA verification tool; candidates that fail are discarded, and only secure results are reported. They report reductions of up to about 15% in power and 10% in area on single gadgets, with many gadgets improving little or getting worse, and a 12% power and 7% area reduction in a 32-bit AES core after replacing one gadget. They also find that conventional CAD steps (technology mapping, FRAIG rewriting, retiming) sometimes give far larger gains on gadgets, up to about 60%, but not systematically, which is the situation Finding 1 explains for equality saturation. Theirs is the closest work to ours in spirit, since it also searches over equivalent masked circuits and keeps only secure ones. It differs in what is searched and how. It works below the Boolean level, on library cells of one gadget, and does not change randomness or latency; we work on the Boolean structure of whole programs. It samples circuits with a randomised search, whereas we saturate an e-graph (every equivalent form, not a sample), add a value-level representation for re-sharing, and show which extraction choices are unsafe. Its security check is applied to the final candidates rather than to every individual in the search, whereas our checks run inside extraction. The two are complementary: a gadget chosen by our optimiser could be passed to theirs.

What we claim. We do not claim to be the first to show that optimisers break masking, the first to use e-graphs on masked circuits, the first to search over equivalent masked circuits under a security check, or the first to remove refreshes or choose gadgets. What is new is the analysis and repair inside equality saturation. Since this is a "nobody has done it" claim (we searched and found nothing), we rate our confidence in this overall claim as medium (the narrower congruence claim of Appendix B as medium-high).

XIII. SCOPE AND LIMITATIONS

- **Scope:** one probe, straight-line circuits over single-bit wires, without loops; no sharing of random bits between gadgets. The compiler uses two shares; Sec. III-A extends only the gadget-level measurement of Finding 1 to $d = 3, 4$, checked against VerifMSI.
- **Savings:** only where the starting circuit has waste; tight designs do not improve.
- **Scale:** checking each candidate is exact only among the k cheapest options, and slow on larger circuits; the 0-of-240 result is a sample.
- **Glitches:** the PINI result and the generated Verilog assume no glitches.
- **Hardware:** no hardware test. The emitted Verilog is synthesised for area and simulated for function on all twelve programs, and VerifMSI checked its glitch-free logic, but no glitch-extended or simulation-based leakage tool was run on it, and it assumes no glitches.
- **Comparison:** a head-to-head against AGEMA and COMPRESS on twelve small programs only, on one machine.

- **Proofs:** written by hand, with a bounded exhaustive check for the leak-cut proposition, not machine-checked.

XIV. Conclusion

Compiler optimisers promise to keep a program's meaning, and masked hardware needs more than that. Left to minimise cost, equality saturation turns secure masked circuits into cheaper ones that leak, and for the two-share ISW gadget this happens by construction rather than by chance. Checking security when the circuit is chosen fixes this at the wire level. The designers' own tricks need a second, value-level e-graph, where choosing one node per class can miss every secure circuit, so the circuit is chosen term by term or through PINI. The result, seqc, removes waste where there is some and returned no leaking circuit on any program we ran in the glitch-free model, and every security verdict about our circuits has been confirmed independently. **For secure hardware, giving the same answer is not enough: the optimiser must also keep the security.**

Acknowledgment

Portions of the code, scripts and this report were prepared with the assistance of an AI system (Claude, Anthropic). The code and data are available at https://github.com/wizard-28/secureeqsat.

Appendix A
Proofs

Throughout, a secret s is shared as $(s_0, s_0 \oplus s)$ with s_0 uniformly random, and all fresh random bits are independent. A wire leaks if what a probe sees on it (its value, or under glitches everything feeding it) has a distribution that depends on the secrets.

Proof of Proposition 3. (i) R3 merges $\mathsf{refresh}(z)$ into the class of z, so $\mathsf{isw}(z, \mathsf{refresh}(z))$ and $\mathsf{isw}(z, z)$ become the same node. (ii) In $\mathsf{isw}(z, z)$ the wire $z_0 \wedge (z_0 \oplus z)$ equals z_0 when $z = 0$ and 0 when $z = 1$, so it leaks. (iii) In $\mathsf{isw}(z, \mathsf{refresh}(z))$, with $y = (z_0 \oplus r', z_1 \oplus r')$, each product $z_i y_j$ is the AND of two values that, together, are a reshuffled copy of the uniform pair (z_0, r'). It is therefore 1 with probability $1/4$ for either value of z. Every other wire is uniform: the input wires because they are shares, and the rest because each contains the gadget's random bit r exactly once. (iv) With R3 alone, z's class holds z's own nodes plus the refresh loop. A one-node-per-class extraction cannot follow the loop, so both inputs become the same wire and the result is $\mathsf{isw}(z, z)$. (v) Term extraction can follow the loop at one input only, and returns the secure term (cost 14 against 11). □

Proof of Proposition 4. Let w be a leaking wire in the lowering of a piece s of a larger term t. A piece is a subterm taken with all of its inputs, and every input of t is shared once, with fresh uniform shares, so the variables that a wire inside s depends on have the same joint distribution in s alone and in t, so putting a piece into a context cannot change what its own wires reveal. Lowering builds t from its pieces, so the lowering of t contains a wire w' that computes the same function as w, up to renaming its random bits, which does not change any distribution. Everything w' depends on lies inside s, because the rest of t only uses s's outputs. So a probe on w' sees exactly what a probe on w sees, and w' leaks too. As a sanity check we also enumerated every term over two inputs with up to three operators: all 2,204 leaking terms stay leaking in all 26 one-step contexts each (57,304 checks, no counterexample). This is a bounded check, not a machine-checked proof. □

Appendix B
Search Record for the Congruence Claim

The claim that re-sharing is not a congruence of the wire-term algebra, and so cannot be an e-graph equality, is a negative novelty claim, so we record how it was checked. The search was run on 25 October 2026 with a web-search tool restricted to a US index, by four independent search passes told to try to refute the claims. The queries bearing on this claim were:

- *Semantic Foundations of Equality Saturation Suciu Wang Zhang ICDT 2025 congruence*
- *e-graphs with contexts OR "colored e-graphs" Singher Itzhaky conditional equalities non-congruence*
- *equality saturation directed/non-symmetric/refinement relations not congruence*
- *equality saturation masked implementation side-channel countermeasure e-graph optimization masking gadgets*
- *"e-graph" OR "equality saturation" "masked" implementation threshold implementation rewriting side-channel*
- *equality saturation e-graph masking probing security side-channel*
- *masked circuit optimization e-graph equality saturation probing security*

We repeated the search on 2 November 2026 with fresh queries and found nothing new:

- *e-graph equality saturation masked circuits refresh re-sharing not a congruence probing security*
- *equality saturation rewrite rules that are not congruences unsound e-graph "randomness" OR "probabilistic" OR "shares" masked implementation*
- *"equality saturation" OR "e-graph" "side-channel" masking countermeasure rewrite security-preserving optimization compiler 2025 2026*
- *superVer verifying probabilistic independence of systems of expressions using equality saturation EGRAPHS 2024*

The only source that uses e-graphs for a masking-related property is superVer [15]; its abstract describes verification of probabilistic independence and does not mention refreshing, rewrites that change randomness, or congruence, but only the workshop abstract of this work in progress is available (we found no longer version), so it may say more. The other hit was an SMT-based masking synthesiser [26], which does not concern re-sharing. No result argues that re-sharing is non-congruent for e-graphs. The nearest sources are the semantics

of equality saturation as a partial congruence [9] and colored e-graphs [27], where each colour is its own congruence layer; neither mentions masking. We rate this claim medium-high, not certain; the overall novelty claim in Sec. XII is rated medium. Several PDFs could not be fetched, so some sources were judged from abstracts or search snippets, and the search tool is snippet-based, so a recent workshop paper could be missed. A later search for the project's title and tool name found nothing, and a search for gadget-level optimisation found the work of Koblah et al. [24], which we read in full; it works on library cells and does not address congruence. The full record, with the reading status of each source, is in `docs/adversarial-search-6.md` of the repository.

References

[1] M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, and P. Panchekha, "egg: Fast and extensible equality saturation," *Proc. ACM Program. Lang.*, vol. 5, no. POPL, 2021.

[2] Y. Zhang, Y. R. Wang, O. Flatt, D. Cao, P. Zucker, E. Rosenthal, Z. Tatlock, and M. Willsey, "Better together: Unifying Datalog and equality saturation," *Proc. ACM Program. Lang.*, vol. 7, no. PLDI, 2023.

[3] Y. Ishai, A. Sahai, and D. Wagner, "Private circuits: Securing hardware against probing attacks," in *CRYPTO 2003*, LNCS vol. 2729, pp. 463–481, 2003.

[4] H. Groß, S. Mangard, and T. Korak, "Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order," IACR Cryptol. ePrint Arch., Rep. 2016/486, 2016.

[5] G. Cassiers, B. Grégoire, I. Levi, and F.-X. Standaert, "Hardware private circuits: From trivial composition to full verification," *IEEE Trans. Comput.*, 2020.

[6] S. Faust, V. Grosso, S. Merino Del Pozo, C. Paglialonga, and F.-X. Standaert, "Composable masking schemes in the presence of physical defaults and the robust probing model," *IACR TCHES*, vol. 2018, no. 3, pp. 89–120, 2018.

[7] S. Mangard, T. Popp, and B. M. Gammel, "Side-channel leakage of masked CMOS gates," in *CT-RSA 2005*, LNCS vol. 3376, pp. 351–365, 2005.

[8] G. Cassiers and F.-X. Standaert, "Trivially and efficiently composing masked gadgets with probe isolating non-interference," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 2542–2555, 2020.

[9] D. Suciu, Y. R. Wang, and Y. Zhang, "Semantic foundations of equality saturation," in *ICDT 2025*, LIPIcs vol. 328, 2025.

[10] S. Belaïd, P.-É. Dagand, D. Mercadier, M. Rivain, and R. Wintersdorff, "Tornado: Automatic generation of probing-secure masked bitsliced implementations," in *EUROCRYPT 2020*, 2020.

[11] D. Knichel, A. Moradi, N. Müller, and P. Sasdrich, "Automated generation of masked hardware," *IACR TCHES*, vol. 2022, no. 1, pp. 589–629, 2022.

[12] Q. L. Meunier and A. Taleb, "VerifMSI: Practical verification of hardware and software masking schemes implementations," in *SECRYPT 2023*, pp. 520–527, 2023.

[13] M. Gourjon, M. Orlt, P. Pauls, and A. Treff, "eVer: Universal and automated verification of side-channel security for additive, inner product, polynomial and general code-based masking," IACR Cryptol. ePrint Arch., Rep. 2026/208, 2026.

[14] N. Sarma, A. S. Thakur, and C. Karfa, "MaskedHLS: Domain-specific high-level synthesis of masked cryptographic designs," arXiv:2407.11806, 2024, preprint.

[15] A. Treff, P. Pauls, M. Orlt, and M. Gourjon, "superVer: Verifying probabilistic independence of systems of expressions using equality saturation," work-in-progress abstract, *EGRAPHS 2024*, 2024.

[16] J. Feldtkeller, D. Knichel, P. Sasdrich, A. Moradi, and T. Güneysu, "Randomness optimization for gadget compositions in higher-order masking," *IACR TCHES*, vol. 2022, no. 4, pp. 188–227, 2022.

[17] G. Cassiers, B. Gigerl, S. Mangard, C. Momin, and R. Nagpal, "Compress: Generate small and fast masked pipelined circuits," *IACR TCHES*, vol. 2024, no. 3, pp. 500–529, 2024.

[18] N. Sarma, T. Patidar, N. Brahamanya, and C. Karfa, "TETRIS: Automated design space exploration of randomness–latency trade-offs in masked hardware," IACR Cryptol. ePrint Arch., Rep. 2026/1306, 2026.

[19] G. Barthe, S. Belaïd, G. Cassiers, P.-A. Fouque, B. Grégoire, and F.-X. Standaert, "maskVerif: Automated verification of higher-order masking in presence of physical defaults," in *ESORICS 2019*, pp. 300–318, 2019.

[20] G. Barthe, S. Belaïd, F. Dupressoir, P.-A. Fouque, B. Grégoire, P.-Y. Strub, and R. Zucchini, "Strong non-interference and type-directed higher-order masking," in *ACM CCS 2016*, pp. 116–129, 2016.

[21] D. Knichel, P. Sasdrich, and A. Moradi, "SILVER – Statistical independence and leakage verification," in *ASIACRYPT 2020*, LNCS, pp. 787–816, 2020.

[22] C. Chen, G. Hu, D. Zuo, C. Yu, Y. Ma, and H. Zhang, "E-Syn: E-graph rewriting with technology-aware cost functions for logic synthesis," in *Proc. DAC*, 2024, arXiv:2403.14242.

[23] Dilip Kumar S. V., B. Gierlichs, and I. Verbauwhede, "A graph-theoretic framework for randomness optimization in first-order masked circuits," IACR Cryptol. ePrint Arch., Rep. 2025/2102, 2025.

[24] D. S. Koblah, D. M. Mehta, N. Poch, M. Hashemi, F. Ganji, and D. Forte, "Lightweight masked hardware gadgets through verification-aware optimization," IACR Cryptol. ePrint Arch., Rep. 2025/1190, revised Aug. 2026, preprint (first posted as "Towards AI-driven optimization of robust probing model-compliant masked hardware gadgets using evolutionary algorithms").

[25] N. T. Courtois, D. Hulme, and T. Mourouzis, "Solving circuit optimisation problems in cryptography and cryptanalysis," preprint hosted by the IMA, 2011.

[26] H. Eldib and C. Wang, "Synthesis of masking countermeasures against side channel attacks," in *Computer Aided Verification (CAV)*, 2014.

[27] E. Singher and S. Itzhaky, "Colored e-graph: Equality reasoning with conditions," arXiv:2305.19203, 2023, preprint.

[28] Fraunhofer AISEC, study on the security impact of hardware design synthesis, commissioned by the German Federal Office for Information Security (BSI), publication page at bsi.bund.de (the summary page was read, not the study).